# 권한, 보안과 효과 통제

모델의 제안과 실제 운영체제 효과 사이에 어떤 판단이 놓이며, 경로·shell·hook·원격 정책이 그 판단을 어떻게 바꾸는지 읽는다

이 권을 마치면 permission dialog를 안전성 전체로 오해하지 않고, authority·validation·isolation·audit가 서로 보완하는 구조를 설명할 수 있다.

**STUDY SNAPSHOT**
Claude Code v2.1.88
2ca5ddabfed5

제3자 복원 소스 기반
한국어 · Guided Edition

# 오늘 우리는 이 코드의 어느 부분을 함께 보는가

코딩 에이전트의 위험은 모델이 틀릴 수 있다는 사실보다 틀린 제안이 넓은 효과로 번역될 수 있다는 데 있다. 이 저장소는 rule, mode, hook, filesystem 검사, shell 분석, sandbox, managed settings를 겹쳐 놓는다. 우리는 레이어 수를 칭찬하는 대신 각 레이어가 막는 실패와 막지 못하는 실패를 구분한다.

| 38 | 13,177 | 50 | 126 |
|---|---|---|---|
| 분류된 TS/TSX 파일 | 물리 코드 줄 | async 함수 | tool use 표기 |

CHAPTER 01

## 권한은 context와 rule precedence로 계산된다

allow/ask/deny 결과는 버튼 하나가 아니라 여러 출처의 규칙, 현재 mode, 도구별 matcher를 합성한 결정이다.

CHAPTER 02

## 파일 경로는 문자열이 아니라 보안 객체다

정규화, symlink, UNC, workspace boundary를 고려하지 않으면 눈에 보이는 경로와 실제 effect 대상이 달라진다.

CHAPTER 03

## Shell 안전성은 해석과 격리를 겹친다

명령 문자열 분석은 사용자의 의도를 설명하고, sandbox는 분석이 틀렸을 때 효과 반경을 줄인다. 어느 하나도 다른 하나를 대체하지 않는다.

CHAPTER 04

## Hooks와 원격 정책은 제어 평면을 확장한다

로컬 permission 외에도 hook, managed settings, feature killswitch가 실행을 바꾼다. 이 경로들의 authority와 failure mode를 따로 보아야 한다.

분포 수치는 중요도 점수가 아니다. 이 권이 직접 펼쳐보는 코드 표면의 규모와 언어 메커니즘 밀도를 가늠하기 위한 탐색 표지다.

01

# 권한은 context와 rule precedence로 계산된다

allow/ask/deny 결과는 버튼 하나가 아니라 여러 출처의 규칙, 현재 mode, 도구별 matcher를 합성한 결정이다.

**이 장에서 실제로 펼쳐볼 경로**

src/utils/permissions/** · src/types/permissions.ts · src/Tool.ts

# ToolPermissionContext는 현재 authority의 snapshot이다

mode, rules by source, working directories, UI 가능성이 한 판단 환경을 만든다.

Tool.ts의 permission context에는 현재 mode와 규칙뿐 아니라 background agent처럼 prompt를 띄울 수 없는 조건도 들어 있다. 같은 tool input이 interactive main session에서는 ask, background에서는 auto-deny가 될 수 있다.

이 context를 전역에서 즉석 조회하면 한 실행 중 규칙이 바뀌어 판단 근거가 흔들린다. 반대로 오래된 snapshot을 끝까지 쓰면 사용자 변경을 반영하지 못한다. 어느 단위에서 snapshot을 고정하는지가 중요하다.

권한 결과와 함께 적용된 rule source와 mode를 남겨야 audit가 가능하다. 단순 boolean은 왜 허용됐는지, 재현 가능한지 설명하지 못한다.

---

**스터디에서 확인할 질문**

한 tool call 중 permission setting이 바뀌면 현재 호출과 다음 호출 중 어디부터 새 정책을 적용해야 하는가?

# deny, ask, allow는 위험 점수의 세 구간이 아니다

각 결과는 다른 authority 의미와 후속 protocol을 가진다.

deny는 정책상 불가능, ask는 현재 context만으로 권한 부족, allow는 조건 충족을 뜻한다. ask를 높은 위험의 deny로 보거나 allow를 안전 보증으로 읽으면 다른 검증 층이 사라진다.

permission decision에는 updatedInput, reason, userModified 같은 정보도 따라갈 수 있다. 사용자가 경로를 바꿔 승인했다면 원래 모델 제안이 아니라 수정된 제안이 실행되고 기록되어야 한다.

UI는 세 결과를 색으로 표현할 수 있지만 SDK와 background agent는 같은 의미를 다른 방식으로 처리한다. domain decision과 presentation을 분리해야 한다.

### Boolean이 아닌 permission result algebra

분석용 TypeScript — 실제 구현의 제어 구조를 축약·재작성

```
type Decision =
  | { behavior: "allow"; updatedInput?: Input }
  | { behavior: "ask"; reason: Reason; preview?: View }
  | { behavior: "deny"; reason: Reason }

switch (decision.behavior) { /* exhaustive */ }
```

1. allow
   안전하다는 뜻이 아니라 현재 authority 조건이 충족되었다는 뜻이다.
2. ask
   사람/host의 추가 결정을 기다리는 protocol state다.
3. deny
   정책상 실행하지 않으며 같은 id의 관찰을 남겨야 한다.

**언어·런타임 안쪽에서는** discriminated union은 각 결과가 가져야 할 payload를 강제한다. number score 하나로 줄이면 authority 의미와 후속 행동이 사라진다.

### 스터디에서 확인할 질문

사용자가 input을 수정해 허용한 경우 모델에게는 원안과 수정안 중 무엇을 tool result로 알려야 하는가?

# 규칙 출처의 precedence가 조직 정책을 실제로 만든다

local allow가 managed deny를 덮을 수 있는지 여부는 파일 저장 위치보다 중요한 보안 결정이다.

permissions 코드는 여러 source의 rule을 모아 매칭한다. 조직 관리 설정, 프로젝트 설정, 사용자 설정, 세션 선택이 충돌할 수 있으므로 순서와 non-overridable 범위를 명시해야 한다.

규칙이 문자열 배열로만 보이면 충돌 이유가 숨는다. source와 scope를 유지해야 "이 deny는 조직에서 왔으므로 local override 불가" 같은 결정을 설명할 수 있다.

새 설정 source를 추가할 때 기존 배열에 끼워 넣지 말고 authority relationship을 먼저 정의하자. precedence test는 보안 회귀를 잡는 핵심이다.

**Effective permission을 만드는 source hierarchy**

| 01 | → | 02 | → | 03 | → | 04 |
|---|---|---|---|---|---|---|
| **Managed** | | **Project** | | **User** | | **Session** |
| 조직이 강제하는 non-overridable floor | | 저장소 범위 rule | | 개인 기본값 | | 현재 prompt에서의 일시 결정 |

**이 그림이 감추지 않는 것** 단순 object spread가 아니라 source-aware merge여야 한다. 값과 함께 provenance를 유지해야 UI와 audit가 "왜"를 설명한다.

### 스터디에서 확인할 질문

프로젝트 저장소 안의 설정이 사용자 홈 설정보다 높은 우선순위를 가지면 어떤 공격 경로가 생길 수 있는가?

# permission mode는 정책 집합을 선택하는 상태다

default, plan, auto, bypass 같은 mode는 같은 rule evaluator의 옵션 이상이다.

mode는 어떤 효과를 기본 허용하고 언제 classifier나 사용자 prompt를 기다리는지 바꾼다. plan mode에서 write를 막는 규칙과 bypass mode의 killswitch는 서로 다른 목적을 가진다.

mode 전환은 UI preference가 아니라 authority transition이다. 이전 mode를 저장하고 복원하는 필드가 있는 이유도 nested plan entry처럼 상태가 겹칠 수 있기 때문이다.

전환을 command handler에만 구현하지 말고 현재 mode, 이전 mode, 전환 원인, 허용된 successor를 명시적으로 모델링하면 잘못된 복원을 줄일 수 있다.

---

**스터디에서 확인할 질문**

모델이 plan mode 진입을 제안할 때와 사용자가 직접 mode를 바꿀 때 authority 차이는 어떻게 기록해야 하는가?

02

# 파일 경로는 문자열이 아니라 보안 객체다

정규화, symlink, UNC, workspace boundary를 고려하지 않으면 눈에 보이는 경로와 실제 effect 대상이 달라진다.

**이 장에서 실제로 펼쳐볼 경로**

src/utils/permissions/filesystem.ts · src/tools/FileEditTool/** · src/utils/file.ts

# normalize가 matcher보다 먼저 와야 한다

상대 경로, ~, separator 차이를 그대로 비교하면 같은 대상이 여러 문자열로 나타난다.

FileEditTool은 hook과 allowlist가 우회되지 않도록 observable input의 file_path를 확장한다. Windows와 POSIX separator 차이도 read cache lookup과 permission match를 어긋나게 할 수 있다.

정규화는 편의 변환이 아니라 identity 결정이다. 하지만 symlink를 resolve할지, 사용자가 입력한 경로를 보존할지는 목적에 따라 다르다. 표시용 경로와 authority 판단용 canonical path를 분리할 필요가 있다.

경로 helper를 공통화할 때 모든 consumer가 같은 normalization을 원하는지 확인하자. git path, UI label, filesystem target은 서로 다른 representation을 가질 수 있다.

---

스터디에서 확인할 질문

`project/link -> /outside` symlink를 workspace 내부 경로로 허용하면 실제 effect boundary는 어디가 되는가?

# UNC 차단은 검사를 위한 접근도 효과가 될 수 있음을 보여 준다

존재 여부 확인만 해도 네트워크 인증이 발생할 수 있다.

Windows UNC path에 대해 `existsSync`를 호출하면 SMB 인증이 유발되어 credential leak 가능성이 생긴다. 그래서 Edit validation은 일부 filesystem operation을 permission 판단 전 생략한다.

이 사례는 “validation은 side-effect free”라는 가정을 깨뜨린다. 외부 resource의 metadata 조회, DNS, symlink resolution도 관찰 가능한 효과가 될 수 있다.

새 validator를 작성할 때 입력만 검사하는지, filesystem/network를 건드리는지 표시하자. side effect가 있다면 authority 전 단계에서 실행해도 되는지 별도로 검토해야 한다.

---

**스터디에서 확인할 질문**

파일 preview를 권한 질문에 보여 주기 위해 먼저 읽는 행위는 어떤 경로에서 이미 민감 정보 접근이 되는가?

# working directory boundary는 lexical prefix가 아니다

비슷한 문자열, parent traversal, symlink 때문에 단순 startsWith는 틀릴 수 있다.

filesystem permission 코드는 추가 working directory와 path rule을 다룬다. 경로가 허용 root 아래인지 판단하려면 normalize된 component boundary를 비교하고 platform case sensitivity도 고려해야 한다.

여러 root를 허용하면 authority surface가 넓어질 뿐 아니라 root 사이의 symlink와 mount 관계도 생긴다. 사용자에게 "이 디렉터리를 추가할까요?"라고 묻는 UI는 실제 범위를 정확히 보여 줘야 한다.

작업공간 boundary를 refactor할 때 path library 교체만 하지 말고 adversarial corpus를 유지하자: `../`, prefix collision, junction, case variation, deleted/recreated path.

---

**스터디에서 확인할 질문**

허용된 root가 실행 중 symlink로 교체되면 기존 permission decision은 언제 무효화되어야 하는가?

# TOCTOU는 permission과 effect 사이에도 존재한다

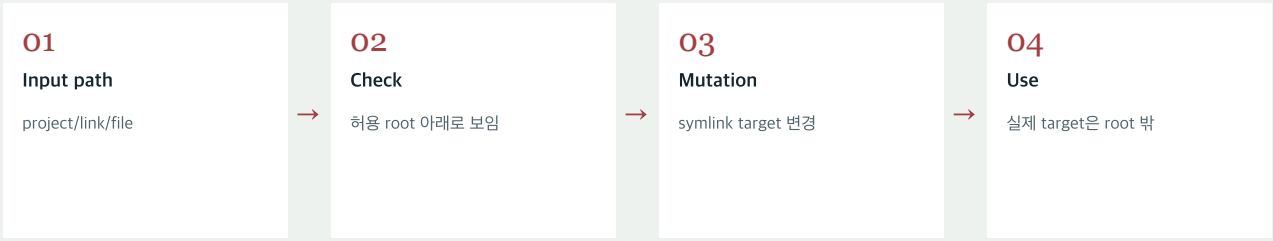
허가할 때 본 대상이 실행할 때 같은 대상이라는 보장이 필요하다.

사용자가 특정 파일 경로를 승인한 뒤 call 전에 symlink target이 바뀔 수 있다. permission matcher가 문자열만 승인하고 executor가 나중에 resolve하면 다른 resource에 효과가 갈 수 있다.

모든 race를 user-space check로 막을 수는 없다. 가능한 경우 file descriptor 기반 작업, no-follow option, atomic primitive를 사용하고 나머지는 sandbox boundary로 제한해야 한다.

중요한 것은 check를 더 많이 쌓는 것이 아니라 check와 use가 같은 identity를 공유하게 하는 것이다. 검증 결과에 canonical target/version token을 담는 설계가 한 방법이다.

**Path 문자열 승인만으로 부족한 이유**




**이 그림이 감추지 않는 것** resource identity를 문자열로만 전달하면 check와 use가 다른 대상을 볼 수 있다. file descriptor·no-follow·version token이 더 강한 연결을 제공한다.

### 스터디에서 확인할 질문

현재 FileEdit 경로에서 permission decision과 write 사이에 target identity가 바뀔 수 있는 지점을 표시해 보라.

**함께 펼쳐볼 자료** src/tools/FileEditTool/FileEditTool.ts:125-180 · src/tools/FileEditTool/FileEditTool.ts:387-480 · CWE-367 TOCTOU race condition

03

# Shell 안전성은 해석과 격리를 겹친다

명령 문자열 분석은 사용자의 의도를 설명하고, sandbox는 분석이 틀렸을 때 효과 반경을 줄인다. 어느 하나도 다른 하나를 대체하지 않는다.

**이 장에서 실제로 펼쳐볼 경로**

src/tools/BashTool/bashSecurity.ts · src/tools/BashTool/bashPermissions.ts · src/utils/sandbox/**

# shell parser가 필요한 이유는 quote 하나 때문이다

공백 split은 인자 경계, substitution, redirect를 이해하지 못한다.

bashSecurity는 command를 구조적으로 보려 하고, 실패 시 보수적 판단을 선택한다. quote와 escape를 잘못 해석하면 harmless text 안의 token을 명령으로 보거나 실제 nested command를 놓칠 수 있다.

완전한 Bash semantics를 재구현하는 것도 위험하다. parser와 실제 shell 버전이 다르면 분석 결과와 실행이 어긋난다. 지원 범위를 명시하고 unknown construct를 안전하게 처리해야 한다.

보안 코드를 리팩터링할 때 clean-code 관점의 함수 분리보다 parser corpus와 differential test가 우선이다. 문자열 정리가 의미를 바꿀 수 있다.

---

**스터디에서 확인할 질문**

command substitution이 중첩된 입력에서 분석기가 모르는 문법을 만나면 ask, deny, sandbox-only 중 무엇이 적절한가?

# 환경 변수는 실행 파일 선택과 라이브러리 로딩을 바꾼다

PATH, LD_*, DYLD_*는 같은 command text의 실제 실행 대상을 교체한다.

bashPermissions는 binary hijack 관련 변수 패턴을 별도로 다룬다. `git status`가 허용되어도 PATH가 공격자 디렉터리를 먼저 가리키면 다른 프로그램이 실행될 수 있다.

따라서 permission rule을 command prefix에만 묶을 수 없다. executable resolution context와 environment mutation도 effect identity의 일부다.

성능을 위해 parsed command를 cache한다면 환경을 key에 포함해야 한다. 그렇지 않으면 이전 안전 판정을 다른 execution context에 재사용한다.

---

**스터디에서 확인할 질문**

명령 승인 UI에 executable의 resolved path를 보여 주는 것이 언제 유용하고 언제 정보 과잉인가?

# sandbox는 authority가 아니라 containment다

허용된 명령도 제한된 filesystem과 network 안에서 실행해 피해 반경을 줄일 수 있다.

Sandbox adapter는 platform/runtime 차이를 감추고 BashTool이 격리 실행을 선택하도록 돕는다. 하지만 sandbox 내부라는 이유로 모든 command를 자동 허용하면 민감 데이터 읽기나 resource exhaustion 같은 위험이 남는다.

권한은 "이 효과를 해도 되는가", sandbox는 "효과가 어디까지 퍼질 수 있는가"를 답한다. 두 질문을 분리하면 sandbox unavailable일 때 fail-open인지 fail-closed인지도 명확해진다.

사용자에게 sandbox 상태를 보여 주는 UI는 실제 adapter capability와 일치해야 한다. 설정이 켜졌다는 표시와 enforcement가 성공했다는 evidence는 다르다.

**Permission과 sandbox가 답하는 서로 다른 질문**

| 층 | 핵심 질문 | 실패 시 결과 |
|---|---|---|
| Permission | 이 subject가 이 effect를 해도 되는가 | 무권한 실행 |
| Validation | 입력이 현재 상태에서 의미 있는가 | 잘못된 대상/명령 |
| Sandbox | 효과가 어디까지 퍼질 수 있는가 | 피해 반경 확대 |
| Audit | 무엇이 실제로 일어났는가 | 복구·설명 불가 |

**구분해야 하는 이유** 여러 control은 중복이 아니라 서로 다른 실패 모델을 가진다. 하나가 있다는 이유로 다른 층을 생략할 수 없다.

### 스터디에서 확인할 질문

sandbox 초기화가 실패했을 때 read-only command와 destructive command를 같은 방식으로 처리해야 하는가?

# destructive warning은 parser 결과의 사용자 projection이다

위험을 발견하는 것과 사람이 이해할 설명으로 바꾸는 것은 다른 단계다.

BashTool 주변에는 destructive command warning과 permission UI가 있다. analyzer가 찾은 위험 요소를 그대로 노출하면 난해하고, 너무 단순화하면 사용자가 실제 범위를 오해한다.

좋은 warning은 대상, 효과 종류, 되돌릴 수 있는지, 왜 기존 rule로 자동 허용되지 않았는지를 보여 준다. “위험할 수 있음”이라는 일반 문구는 decision support가 아니다.

UI 표현을 바꿀 때 analyzer의 severity를 새로 정의하지 말자. presentation은 policy result를 설명하되 authority를 추가하거나 약화하지 않아야 한다.

**스터디에서 확인할 질문**

복수 subcommand 중 하나만 destructive할 때 전체 명령을 어떻게 요약해야 사용자가 승인 범위를 이해할 수 있는가?

04

# Hooks와 원격 정책은 제어 평면을 확장한다

로컬 permission 외에도 hook, managed settings, feature killswitch가 실행을 바꾼다. 이 경로들의 authority와 failure mode를 따로 보아야 한다.

**이 장에서 실제로 펼쳐볼 경로**

src/hooks/** · src/services/remoteManagedSettings/** · src/services/policyLimits/**

# hook은 확장점이지 자동으로 신뢰된 판정자가 아니다

PreToolUse와 PermissionRequest hook은 input과 decision에 영향을 줄 수 있으므로 출처와 실패 정책이 중요하다.

toolExecution은 hook 결과가 updatedInput과 permission decision을 제공하는 경우를 처리한다. 외부 command hook이 allow를 반환했다고 해서 그 command 자체가 신뢰된 authority인지, advisory인지 설정에 따라 달라질 수 있다.

hook timeout, crash, malformed output에서 fail-open하면 우회가 되고 fail-closed하면 availability가 떨어진다. 모든 hook에 같은 실패 정책을 쓰기보다 effect class와 deployment trust를 고려해야 한다.

hook이 무엇을 보았는지도 privacy surface다. 전체 tool input을 전달하면 secret이 외부 script나 service로 나갈 수 있다.

---

**스터디에서 확인할 질문**

사용자 로컬 hook과 조직 관리 hook이 상충할 때 어느 결과가 우선이며 그 이유를 UI에 어떻게 설명할 것인가?

# remote managed settings는 동적 정책 배포 시스템이다

주기적 fetch, cache, security check가 로컬 동작을 원격 상태와 결합한다.

저장소 문서는 1시간 polling과 stale cache fallback을 강조한다. 코드를 읽을 때는 endpoint보다 fetch 자격, signature/trust, cache freshness, 위험 변경 확인, 거부 시 종료 경로를 함께 본다.

원격 설정은 enterprise policy를 빠르게 적용하지만 control-plane outage와 stale policy 문제를 만든다. cache를 쓰면 availability는 높아지지만 revoked rule이 남을 수 있다.

이 기능을 평가할 때 "원격 제어"라는 가치 판단만으로 끝내지 말자. 어떤 설정이 non-overridable인지, 어떤 change가 사용자 확인을 요구하는지, read-back이 있는지를 구체적으로 보아야 한다.

**스터디에서 확인할 질문**

서버가 장기간 불가능할 때 stale managed allow와 stale managed deny 중 어느 쪽을 어떻게 처리해야 하는가?

# feature killswitch는 출시 가변성과 긴급 통제를 함께 담는다

같은 boolean gate라도 실험, cohort, 안전 중단은 서로 다른 운영 의미다.

권한 bypass, auto mode, voice 같은 기능은 환경과 원격 flag 조합으로 활성화된다. flag 이름만 보아 목적을 추정하기 어려운 경우도 있어 call site와 fallback을 따라가야 한다.

killswitch는 사고 대응에 유용하지만 code path가 평소 실행되지 않으면 부패할 수 있다. off 전환이 현재 세션에 즉시 적용되는지 다음 시작부터 적용되는지도 기능마다 다르다.

gate를 문서화할 때 이름 목록보다 owner, default, evaluation time, stale behavior, rollback을 기록하자. 그래야 숨은 configuration이 architecture로 드러난다.

**스터디에서 확인할 질문**

실행 중 auto mode killswitch가 켜졌을 때 진행 중 효과와 다음 tool call을 각각 어떻게 처리해야 하는가?

# 안전성 보고서는 주장보다 반례를 남겨야 한다

코드가 방어를 포함한다는 사실과 실제 위협을 충분히 막는다는 결론은 다르다.

대상 저장소의 자체 문서는 telemetry와 remote control에서 강한 결론을 제시한다. 스터디에서는 해당 symbol, call site, 조건을 확인하되 production cohort와 서버 동작처럼 코드 밖 사실은 unknown으로 남긴다.

보안 분석은 "취약/안전" 이분법보다 threat scenario가 유용하다. 공격자, 자산, entry, precondition, expected control, residual risk를 적으면 코드의 실제 역할과 한계가 보인다.

수정 제안도 control을 늘리는 데서 끝나지 않아야 한다. 사용자 부담, false positive, recovery, audit를 함께 평가해야 운영 가능한 안전성이 된다.

---

**스터디에서 확인할 질문**

FileEdit symlink race와 remote setting stale cache 중 하나를 골라 완전한 threat scenario를 작성해 보라.

# 코드 뒤에서 작동하는 언어·런타임 메커니즘

문법 이름을 외우기보다, 런타임이 실제로 무엇을 저장하고 언제 제어를 넘기는지와 이 프로젝트가 그것을 어떤 책임에 사용하는지를 함께 본다.

## immutable context snapshot

**언어가 제공하는 것**
한 판단에 필요한 mode·rules·roots를 한 값으로 전달한다.

**이 저장소가 쓰는 방식**
tool call이 어떤 authority 환경에서 평가됐는지 고정한다.

**놓치기 쉬운 함정**
오래 유지하면 정책 변경이 반영되지 않고, 매번 읽으면 한 호출 안에서 흔들린다.

## pattern matcher

**언어가 제공하는 것**
정규화된 resource와 rule pattern의 일치 여부를 계산한다.

**이 저장소가 쓰는 방식**
파일 경로와 shell prefix의 allow/deny/ask 후보를 찾는다.

**놓치기 쉬운 함정**
canonicalization 전 문자열 비교는 traversal·separator·symlink를 놓친다.

## result algebra

**언어가 제공하는 것**
allow/ask/deny를 boolean 대신 서로 다른 payload를 가진 variant로 표현한다.

**이 저장소가 쓰는 방식**
reason, updatedInput, userModified를 실행 pipeline에 보존한다.

**놓치기 쉬운 함정**
ask를 deny로 뭉개거나 allow를 안전성 증명으로 오해하기 쉽다.

## reference monitor

**언어가 제공하는 것**
resource effect 직전에 subject 요청과 policy를 중재한다.

**이 저장소가 쓰는 방식**
toolExecution의 공통 permission 단계가 모든 도구를 통과시킨다.

**놓치기 쉬운 함정**
hook·validator·sandbox가 monitor를 우회하는 별도 효과를 만들 수 있다.

## sandbox adapter

**언어가 제공하는 것**
플랫폼별 격리 primitive를 공통 실행 interface 뒤에 둔다.

**이 저장소가 쓰는 방식**
Bash process의 filesystem/network 반경을 줄인다.

**놓치기 쉬운 함정**
격리는 authorization이 아니며 unavailable 시 fallback 정책이 필요하다.

## TOCTOU identity

**언어가 제공하는 것**
check 시점의 resource가 use 시점에도 같은 대상인지 확인하는 문제다.

**이 저장소가 쓰는 방식**
path normalization, symlink와 Edit stale 검사를 함께 보게 한다.

**놓치기 쉬운 함정**
check를 늘려도 같은 file descriptor/version을 공유하지 않으면 race가 남는다.

# 스터디용 코드 탐색 인덱스

파일을 처음부터 끝까지 읽지 않는다. 아래 시작점에서 symbol을 잡고 정의·호출·상태·실패 경로를 왕복한다.

| 시작 경로 | 먼저 볼 표현 | 그 다음에 물을 질문 |
| --- | --- | --- |
| src/types/permissions.ts | context/result variants | authority 의미가 타입에 충분히 남는가 |
| src/utils/permissions/permissions.ts | precedence와 matcher | managed deny를 local allow가 덮을 수 있는가 |
| src/utils/permissions/filesystem.ts | path/root 판정 | lexical path와 real target을 구분하는가 |
| src/tools/BashTool/bashPermissions.ts | prefix rule 추출 | environment와 compound command를 포함하는가 |
| src/tools/BashTool/bashSecurity.ts | parser fallback | unknown syntax가 fail-open되지 않는가 |
| src/utils/sandbox/sandbox-adapter.ts | platform adapter | 실제 enforcement 성공을 확인하는가 |
| src/hooks | permission-related hook | advisory와 authoritative hook이 구분되는가 |
| src/services/remoteManagedSettings/index.ts | poll/cache/apply | stale policy의 availability/security tradeoff는 무엇인가 |

**코드 독해 규칙**

타입 선언만 보고 runtime 검증을 가정하지 않는다. 호출 순서만 보고 authority를 가정하지 않는다. UI 표시만 보고 durable state를 가정하지 않는다. 주석만 보고 production 활성이나 upstream 의도를 확정하지 않는다.

# 하나의 위험한 명령에 대한 control stack 검토

각 방어가 무엇을 아는지, 어디에서 실패하는지, 다음 방어가 무엇을 보완하는지 연결한다.

1. 환경 변수 변경과 redirect를 포함한 compound shell command를 하나 만든다.
2. schema, bash parser, permission rule, user ask, sandbox, process 실행 단계별 입력을 기록한다.
3. parser unknown, hook timeout, sandbox unavailable, managed setting stale 네 실패를 각각 삽입한다.
4. 각 실패에서 fail-open/fail-closed 결과와 사용자에게 보여야 할 설명을 비교한다.
5. 남는 residual risk 하나를 threat scenario 형식으로 작성한다.

> **완료의 기준**
>
> “permission이 있으니 안전하다”가 아니라 여러 control이 서로 다른 실패를 줄이며 남는 위험도 존재한다고 설명할 수 있으면 된다.

**다음 모임에 남길 기록**

다음 모임에는 control 하나를 제거했을 때 어떤 공격 경로가 새로 열리는지 counterfactual을 준비한다.